

Clase 355 — Telemetría para Game Security

Parte: **19 — Seguridad de videojuegos, cheats y anti-cheat** · Fuente principal: documentación de Epic Games y Valve 🕒 Duración estimada: **120 min** · Nivel: **Avanzado**

Objetivo

Comprender y verificar **telemetría para game security** sobre el Game Security Range, conectando vulnerabilidad, abuso controlado, telemetría, evidencia, causa raíz, mitigación y prueba de regresión sin actuar sobre software de terceros.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Explicar** schema y versionado desde la arquitectura y no solo desde la herramienta.
- 2. **Relacionar** identidad sintética y timestamps con una frontera de confianza concreta.
- 3. **Producir** evidencia reproducible sobre correlación y calidad.
- 4. **Evaluar** límites, falsos positivos y consecuencias de minimización y reproducibilidad.
- 5. **Verificar** una mitigación mediante un caso legítimo y uno adversarial.

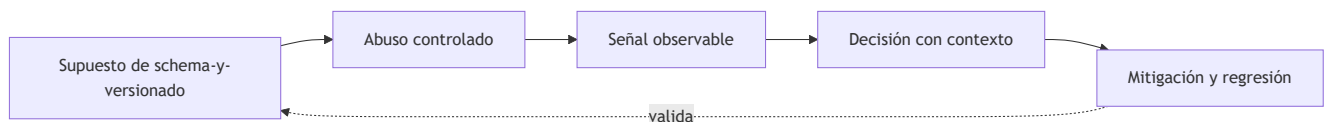
Temas

#	Tema	Evidencia esperada
1	Schema y versionado	Produce una decisión o evidencia verificable
2	Identidad sintética y timestamps	Produce una decisión o evidencia verificable
3	Correlación y calidad	Produce una decisión o evidencia verificable
4	Minimización y reproducibilidad	Produce una decisión o evidencia verificable

Explicación en profundidad

Telemetría útil registra decisión y contexto, no vigilancia indiscriminada. Un evento puede llevar event_id, timestamp, match/session/player sintéticos, entidad, posición, velocidad, aceleración, yaw/pitch, arma, ammo, shot, hit, damage, target, visibilidad, latencia y server decision. No todos los eventos necesitan todos los campos: un schema tipado define obligatoriedad, unidades, nulabilidad y versión.

El tiempo debe declarar origen y semántica; mezclar reloj de cliente con servidor fabrica secuencias falsas. Correlation IDs unen comando, decisión y resultado. Calidad implica completitud, rango, duplicados y drift de instrumentación. Minimización pregunta si cada campo cambia una decisión; reproducibilidad conserva versión, configuración y semilla. Retener menos reduce exposición y obliga a diseñar evidencia suficiente en vez de coleccionar todo.



El diagrama se lee como un bucle de ingeniería, no como una cadena de sanción. El supuesto habilita un abuso dentro del target propio; la señal solo permite formular una hipótesis; la decisión incorpora contexto; la mitigación debe volver al supuesto y demostrar con una regresión que la confianza cambió. Si el último arco no puede verificarse, solo se ocultó el síntoma.

Definiciones y características

- **Señal:** medida observable; característica clave: no prueba por sí sola intención.
- **Evidencia:** señal contextualizada y reproducible; característica clave: trazabilidad.
- **Autoridad:** componente que decide el estado canónico; característica clave: valida intenciones.
- **Causa raíz:** condición sistémica que permitió el incidente; característica clave: su corrección evita recurrencia.

Glosario

Término	Definición en contexto
Schema	Concepto de esta clase aplicado al Game Security Range y a su frontera de confianza.
Identidad	Concepto de esta clase aplicado al Game Security Range y a su frontera de confianza.
Correlación	Concepto de esta clase aplicado al Game Security Range y a su frontera de confianza.
Minimización	Concepto de esta clase aplicado al Game Security Range y a su frontera de confianza.

Herramientas y preparación

- Python 3.12 y el [Game Security Range](#).
- Escenario correspondiente de [SCENARIOS.md](#).
- Dataset sintético documentado; nunca datos personales ni procesos de terceros.
- Prerrequisitos: clases 023–024 (procesos/memoria), 130–134 (RE/debugging), 182/199 (telemetría/detección) y 217 (RCA).

Laboratorio guiado

1. Ejecuta `python -m unittest discover -s tests -v` desde `labs/game-security` y conserva la línea base.
2. Define JSON Schema v1 para un disparo y valida eventos del rango. Documenta unidad, reloj, campos opcionales, retención y consulta que justifica cada dato.
3. Registra modo, comando, decisión, señal, umbral o invariante, semilla y resultado esperado.
4. Formula al menos una explicación legítima alternativa antes de atribuir abuso.
5. Aplica o diseña la mitigación y repite el caso adversarial y un caso normal.
6. Redacta la cadena `síntoma → timeline → evidencia → hipótesis → causa raíz → fix → regresión`.

Ejercicios

1. Explica qué cambia si el cliente controla el dato frente a si solo solicita una acción.
2. Identifica una señal débil y combínala con otra fuente independiente.
3. Diseña un caso límite de latencia, FPS, dispositivo o accesibilidad.
4. Separa prevención, detección, respuesta y gobernanza para este tema.
5. Explica qué información no necesitas recoger y por qué.

Reto verificable

Entrega una ficha con threat boundary, reproducción en el rango, evento de telemetría, decisión explicada, alternativa legítima, causa raíz, mitigación y prueba de regresión.

Criterio de aceptación: otra persona puede ejecutar los comandos con la misma semilla, obtener la evidencia citada y comprobar que la mitigación bloquea el caso adversarial sin romper el caso normal.

Errores comunes

Síntoma	Causa y corrección
La anomalía se trata como culpabilidad	Falta contexto; correlaciona y revisa alternativas legítimas.
El control solo oculta un valor	La autoridad no cambió; valida o deriva el estado en servidor.
El experimento no se reproduce	Faltan semilla, versión, unidades o configuración; regístralas.
El detector castiga lag/FPS	El dataset no estratifica condiciones; evalúa por subpoblación.
Se propone instrumentar terceros	Fuera del alcance; usa exclusivamente el target educativo.

Preguntas frecuentes

¿Una alerta basta para sancionar? No. Es una hipótesis con un nivel de confianza; la consecuencia exige evidencia proporcional, política, auditabilidad y apelación.

¿Ofuscar el cliente resuelve el problema? Puede elevar costo, pero no reemplaza autoridad, invariantes ni minimización de información.

¿Por qué el laboratorio es sintético? Permite controlar verdad, semilla y falsos positivos sin invadir jugadores ni construir una herramienta reutilizable contra terceros.

Referencias

- Epic Games, *Networking Overview* — autoridad y replicación aplicadas a Identidad sintética y timestamps. <https://dev.epicgames.com/documentation/unreal-engine/networking-overview-for-unreal-engine>
- Valve, *Latency Compensating Methods* — predicción, interpolación y límites usados para Minimización y reproducibilidad. https://developer.valvesoftware.com/wiki/Latency_Compensating_Methods_in_Client/Server_In-game_Protocol_Design_and_Optimization

- NIST Privacy Framework 1.0 — minimización, gobernanza y riesgo de datos en la clase 355.
<https://www.nist.gov/privacy-framework>

Clase anterior

Clase 354 — Server-side Anti-Cheat y diseño autoritativo

Siguiente clase

Clase 356 — Detección de aimbot y automatización por comportamiento